

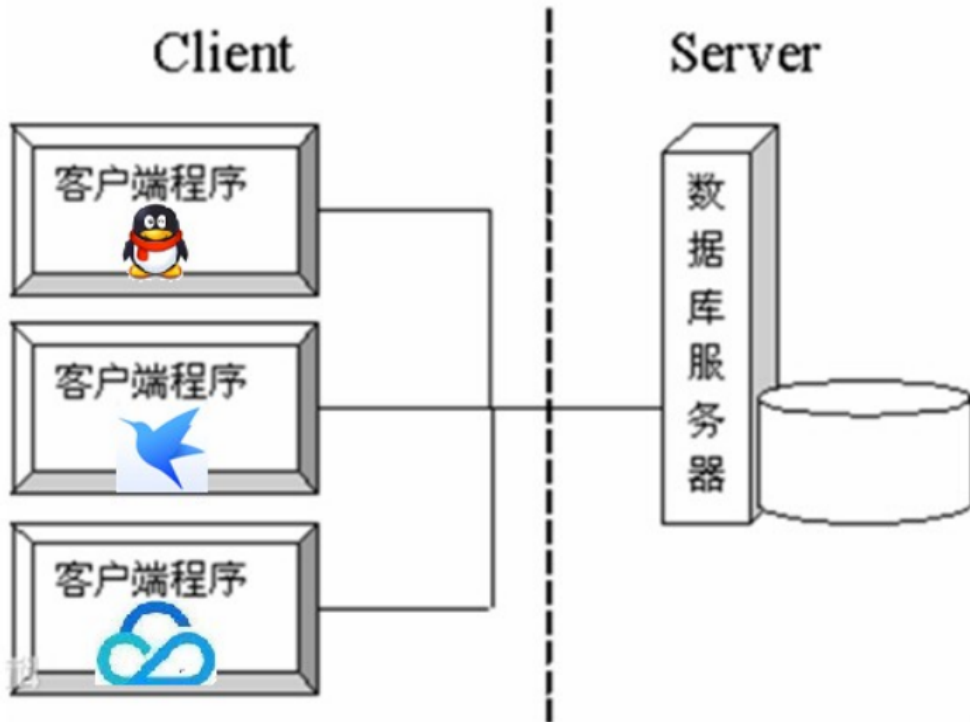
1.网络编程入门

1.1软件结构

网络层面的架构。MVC是代码层面的三层架构

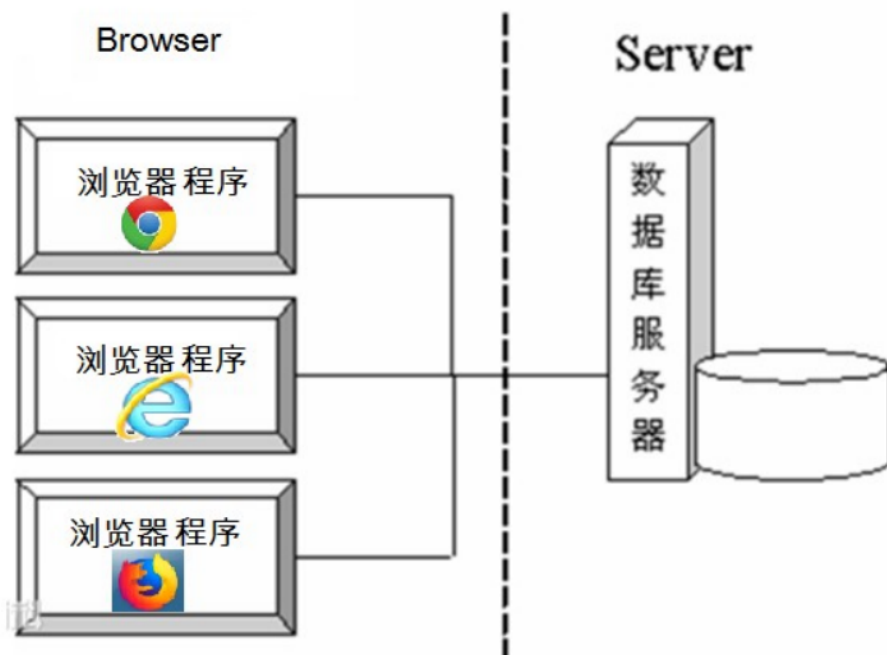
- **C/S结构**：全称为Client/Server结构，是指客户端和服务端结构。常见程序有QQ、迅雷等软件。

客户端升级不方便



- **B/S结构**：全称为Browser/Server结构，是指浏览器和服务端结构。常见浏览器有谷歌、火狐等。

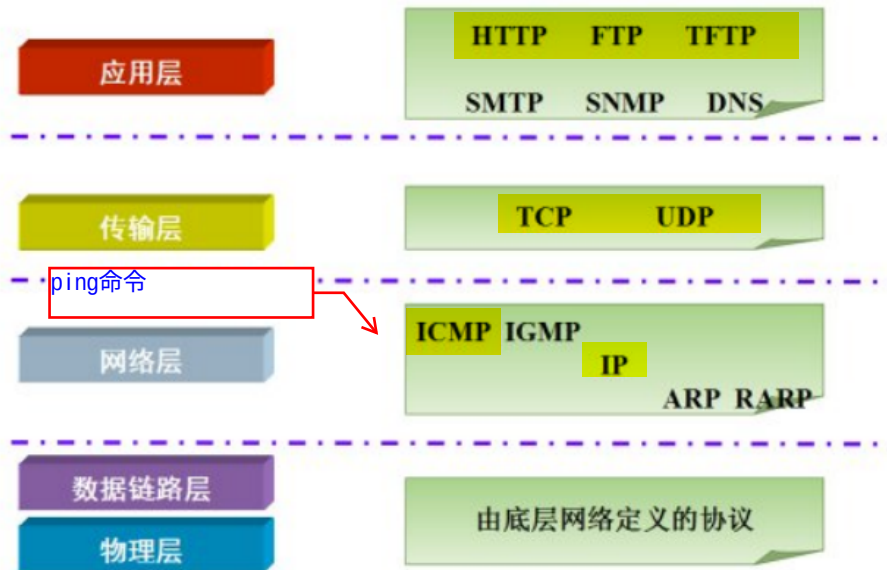
只用考虑服务端，升级方便



两种架构各有优势，但是无论哪种架构，都离不开网络的支持。**网络编程**，就是在一定的协议下，实现两台计算机的通信的程序。

1.2 网络通信协议

- **网络通信协议**：通过计算机网络可以使多台计算机实现连接，位于同一个网络中的计算机在进行连接和通信时需要遵守一定的规则，这就好比在道路中行驶的汽车一定要遵守交通规则一样。在计算机网络中，这些连接和通信的规则被称为网络通信协议，它对数据的传输格式、传输速率、传输步骤等做了统一规定，通信双方必须同时遵守才能完成数据交换。
- **TCP/IP协议**：传输控制协议/因特网互联协议(Transmission Control Protocol/Internet Protocol)，是Internet最基本、最广泛的协议。它定义了计算机如何连入因特网，以及数据如何在它们之间传输的标准。它的内部包含一系列的用于处理数据通信的协议，并采用了4层的分层模型，每一层都呼叫它的下一层所提供的协议来完成自己的需求。



上图中，TCP/IP协议中的四层分别是应用层、传输层、网络层和链路层，每层分别负责不同的通信功能。

链路层：链路层是用于定义物理传输通道，通常是对某些网络连接设备的驱动协议，例如针对光纤、网线提供的驱动。

网络层：网络层是整个TCP/IP协议的核心，它主要用于将传输的数据进行分组，将分组数据发送到目标计算机或者网络。

传输层：主要使网络程序进行通信，在进行网络通信时，可以采用TCP协议，也可以采用UDP协议。

应用层：主要负责应用程序的协议，例如HTTP协议、FTP协议等。

1.3 协议分类

通信的协议还是比较复杂的，`java.net` 包中包含的类和接口，它们提供低层次的通信细节。我们可以直接使用这些类和接口，来专注于网络程序开发，而不用考虑通信的细节。

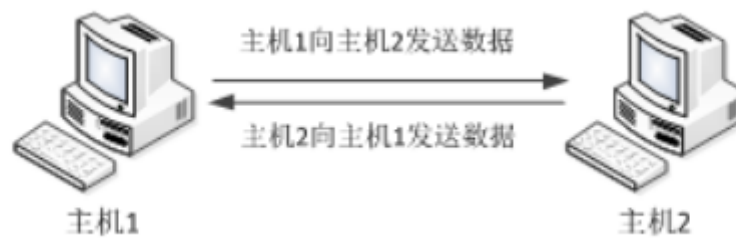
`java.net` 包中提供了两种常见的网络协议的支持：

- **UDP**：用户数据报协议(User Datagram Protocol)。UDP是**无连接**通信协议，即在数据传输时，**数据的发送端和接收端不建立逻辑连接**。简单来说，当一台计算机向另外一台计算机发送数据时，发送端不会确认接收端是否存在，就会发出数据，同样接收端在收到数据时，也不会向发送端反馈是否收到数据。

↑
无连接、性能高。游戏使用的是UDP

由于使用UDP协议消耗资源小，通信效率高，所以通常都会用于音频、视频和普通数据的传输例如视频会议都使用UDP协议，因为这种情况即使偶尔丢失一两个数据包，也不会对接收结果产生太大影响。

但是在使用UDP协议传送数据时，由于UDP的面向无连接性，不能保证数据的完整性，因此在传输重要数据时不建议使用UDP协议。UDP的交换过程如下图所示。



特点:数据被限制在64kb以内，超出这个范围就不能发送了。

数据报(Datagram):网络传输的基本单位

- **TCP**: 传输控制协议 (Transmission Control Protocol)。TCP协议是**面向连接**的通信协议，即传输数据之前，**在发送端和接收端建立逻辑连接**，然后再传输数据，它提供了两台计算机之间可靠无差错的数据传输。

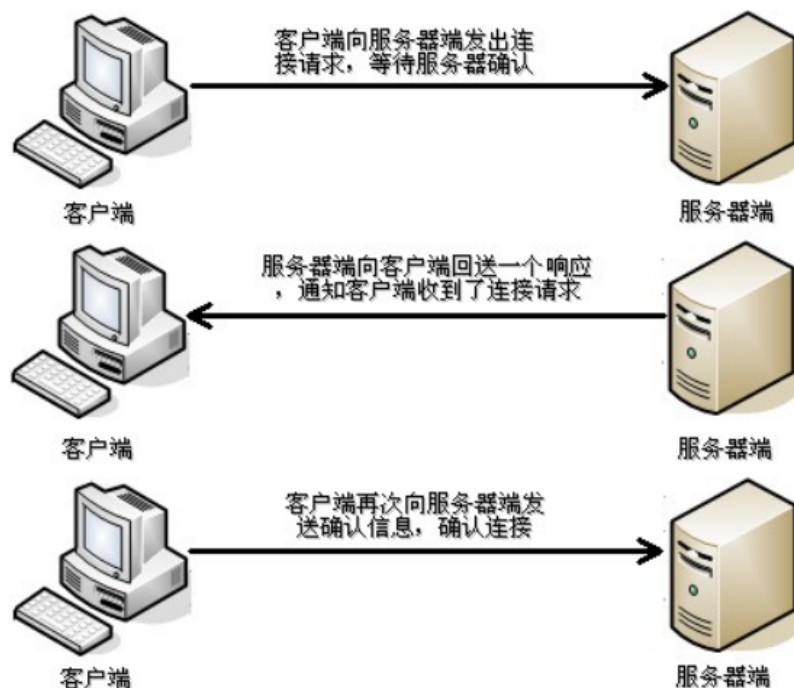


有连接、性能低、双向链接

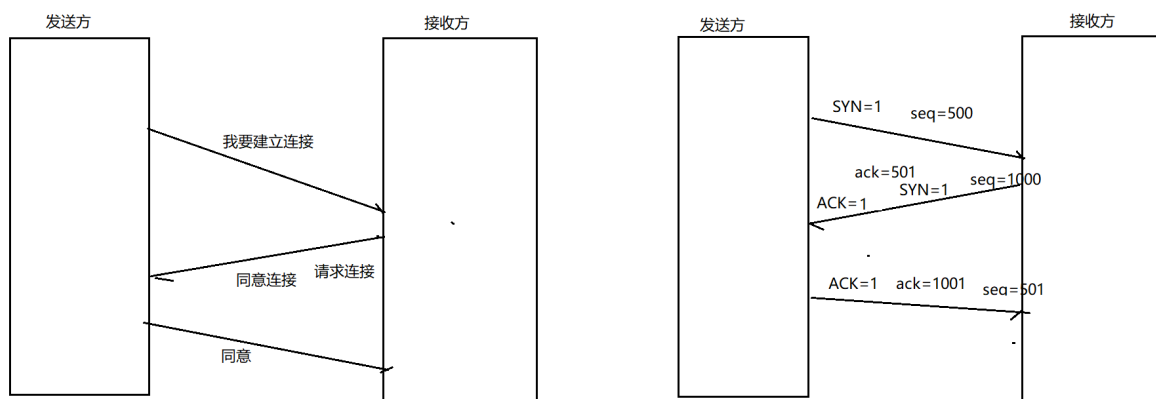
在TCP连接中必须要明确客户端与服务器端，由客户端向服务器端发出连接请求，每次连接的创建都需要经过“三次握手”。

- **三次握手**: TCP协议中，在发送数据的准备阶段，客户端与服务器之间的三次交互，以保证连接的可靠。
 - 第一次握手，客户端向服务器端发出连接请求，等待服务器确认。
 - 第二次握手，服务器端向客户端回送一个响应，通知客户端收到了连接请求。
 - 第三次握手，客户端再次向服务器端发送确认信息，确认连接。整个交互过程如下图所示。

- 第三次握手，客户端再次向服务器端发送确认信息，确认连接。整个交互过程如下图所示。



完成三次握手，连接建立后，客户端和服务端就可以开始进行数据传输了。由于这种面向连接的特性，TCP协议可以保证传输数据的安全，所以应用十分广泛，例如下载文件、浏览网页等。



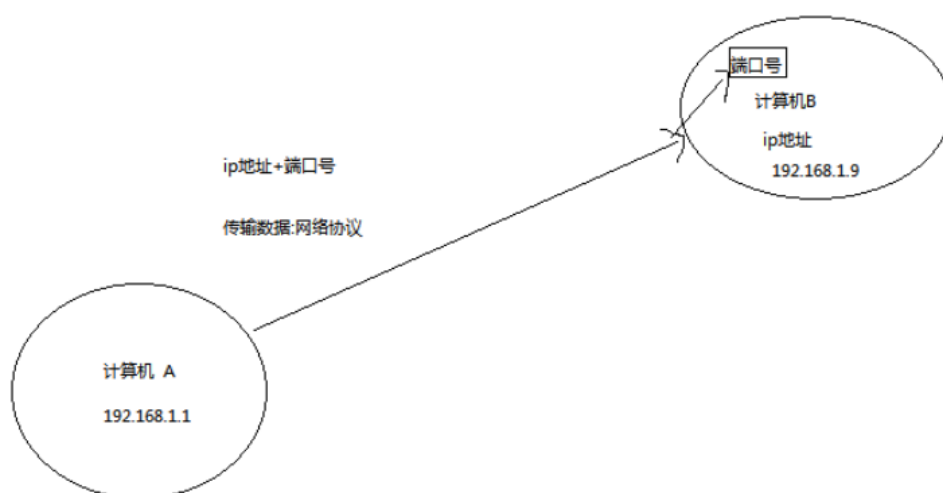
1.4 网络编程三要素

网络编程三要素:

A:IP地址

B:端口

C:协议



协议

- **协议:** 计算机网络通信必须遵守的规则，已经介绍过了，不再赘述。

IP地址

- **IP地址:** 指互联网协议地址 (Internet Protocol Address)，俗称IP。IP地址用来给一个网络中的计算机设备做唯一的编号。假如我们把“个人电脑”比作“一台电话”的话，那么“IP地址”就相当于“电话号码”。

IP地址分类

- **IPv4:** 是一个32位的二进制数，通常被分为4个字节，表示成 a.b.c.d 的形式，例如 192.168.65.100。其中a、b、c、d都是0~255之间的十进制整数，那么最多可以表示42亿个。

- IPv6：由于互联网的蓬勃发展，IP地址的需求量愈来愈大，但是网络地址资源有限，使得IP的分配越发紧张。

为了扩大地址空间，拟通过IPv6重新定义地址空间，采用128位地址长度，每16个字节一组，分成8组十六进制数，表示成ABCD:EF01:2345:6789:ABCD:EF01:2345:6789，号称可以为全世界的每一粒沙子编上一个网址，这样就解决了网络地址资源数量不够的问题。

IP地址：

网络计算机的唯一标识

计算机底层只能识别二进制数据，所以我们的ip地址应该是一个二进制数据

192.168.1.100

11000000 10101000 000000001 01100100

4个字节 一个字节8位

一个字节能标识的十位数 0~255

IP地址的组成

网络号段+主机号段

A类：第一个号段网络号段+后三段主机号段

B类：前两个号段网络号段+后两段主机号段

C类：前三个号段网络号段+后一个号段为主机号段

0-255 256

IP地址分类：IP地址根据网络号和主机号来分，分为A、B、C三类及特殊地址D、E。全0和全1的都保留不用。

A类：(1.0.0.0-126.0.0.0)（默认子网掩码：255.0.0.0或0xFF000000）第一个字节为网络号，后三个字节为主机号。该类IP地址的最前面为“0”，所以地址的网络号取值于1~126之间。一般用于大型网络。

B类：(128.0.0.0-191.255.0.0)（默认子网掩码：255.255.0.0或0xFFFF0000）前两个字节为网络号，后两个字节为主机号。该类IP地址的最前面为“10”，所以地址的网络号取值于128~191之间。一般用于中等规模网络。

C类：(192.0.0.0-223.255.255.0)（子网掩码：255.255.255.0或0xFFFFFF00）前三个字节为网络号，最后一个字节为主机号。该类IP地址的最前面为“110”，所以地址的网络号取值于192~223之间。一般用于小型网络。

D类：是多播地址。该类IP地址的最前面为“1110”，所以地址的网络号取值于224~239之间。一般用于多路广播用户[1]。

E类：是保留地址。该类IP地址的最前面为“1111”，所以地址的网络号取值于240~255之间。

在IP地址3种主要类型里，各保留了3个区域作为私有地址，其地址范围如下：

A类地址: 10.0.0.0~10.255.255.255
B类地址: 172.16.0.0~172.31.255.255
C类地址: 192.168.0.0~192.168.255.255

回送地址: 127.0.0.1。也是本机地址, 等效于localhost或本机IP。一般用于测试使用。例如: ping 127.0.0.1来测试本机TCP/IP是否正常。

IP+端口 => socket (套接字)

常用命令

- 查看本机IP地址, 在控制台输入:

ipconfig

windows: ipconfig
Linux: ifconfig

- 检查网络是否连通, 在控制台输入:

ping 空格 IP地址
ping 220.181.57.216

特殊的IP地址

- 本机IP地址: 127.0.0.1、localhost。

端口号

网络的通信, 本质上是两个进程(应用程序)的通信。每台计算机都有很多的进程, 那么在网络通信时, 如何区分这些进程呢?

ip地址 <==> 设备的身份证
端口号 <==> 程序的身份证

如果说IP地址可以唯一标识网络中的设备, 那么端口号就可以唯一标识设备中的进程(应用程序)了。

- 端口号:** 用两个字节表示的整数, 它的取值范围是0~65535。其中, 0~1023之间的端口号用于一些知名的网络服务和应用, 普通的应用程序需要使用1024以上的端口号。如果端口号被另外一个服务或应用所占用, 会导致当前程序启动失败。

利用 协议 + IP地址 + 端口号 三元组合, 就可以标识网络中的进程了, 那么进程间的通信就可以利用这个标识与其它进程进行交互。

2.UDP编程

2.1 InetAddress

查看端口号
windows: netstat -ano | findstr 端口号
Linux: netstat -nlt | grep 端口号

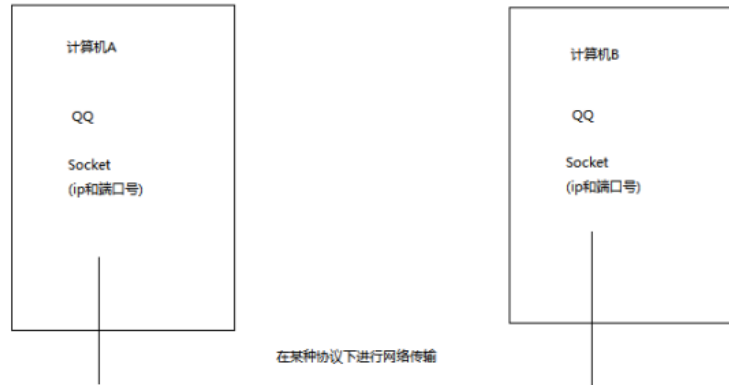
```
//static InetAddress getByName(String host)
//在给定主机名的情况下确定主机的 IP 地址。
InetAddress address = InetAddress.getByName("127.0.0.1");

//获取ip地址
System.out.println(address.getHostAddress());
```

2.2 udp发送数据

Socket: 网络套接字
Socket编程

Socket包含:ip地址+端口号



```
public class SendDemo {

    public static void main(String[] args) throws Exception {

        //1.创建数据包套接字 udp对象
        DatagramSocket datagramSocket = new DatagramSocket();

        //准备ip地址
        InetAddress inetAddress = InetAddress.getByName("127.0.0.1");

        //2.准备数据包

        byte[] bytes = "hello,强哥".getBytes();

        DatagramPacket dp=new DatagramPacket(bytes, bytes.length, inetAddress,
20881);

        //3.发送数据包
        datagramSocket.send(dp);

        //4.释放资源
        datagramSocket.close();

    }

}
```

2.3 udp接收数据

```
public class ReceiveDemo {  
  
    public static void main(String[] args) throws Exception {  
  
        //1.创建接收端的udp对象  
        DatagramSocket ds=new DatagramSocket(20881);  
  
        //2.准备数据包，接收的容器  
        byte[] bys=new byte[1024];  
        DatagramPacket dp=new DatagramPacket(bys, bys.length);  
  
        //3.接收数据  
        ds.receive(dp); //阻塞  
  
        //4.获取发送端的ip地址  
        InetAddress address = dp.getAddress();  
  
        System.out.println("ip地址:"+address.getHostAddress());  
  
        //5.解析数据  
        byte[] data = dp.getData();  
        System.out.println(new String(data,0,dp.getLength()));  
  
        ds.close();  
  
    }  
  
}
```

3.TCP编程

3.1 发送数据

```
// Socket(InetAddress address, int port)
// 1.创建Socket对象
Socket socket = new Socket("127.0.0.1", 8080);

//2.发送数据
//获取字节输出流
OutputStream out = socket.getOutputStream();

out.write("林老师, 我来了".getBytes());

socket.close();
```

3.2 接收数据

```
// ServerSocket(int port)
// 1.创建绑定到特定端口的服务器套接字。
ServerSocket ss = new ServerSocket(8080);

// 2.监听客户端的请求,获取客户端的Socket对象
// Socket accept()
// 侦听并接受到此套接字的连接。
Socket socket = ss.accept();// 阻塞

InputStream in = socket.getInputStream();

InetAddress inetAddress = socket.getInetAddress();

byte[] buffer=new byte[1024];

int len=in.read(buffer);

System.out.println("ip"+inetAddress.getHostAddress()+new String(buffer,
0, len));

socket.close();
ss.close();
```

3.3 服务器反馈

```
// ServerSocket(int port)
// 1.创建绑定到特定端口的服务器套接字。
ServerSocket ss = new ServerSocket(8080);

// 2.监听客户端的请求,获取客户端的Socket对象
// Socket accept()
// 侦听并接受到此套接字的连接。
```

```

Socket socket = ss.accept();// 阻塞

//3.获取客户端的数据 解析数据
InputStream in = socket.getInputStream();

InetAddress inetAddress = socket.getInetAddress();

byte[] buffer=new byte[1024];

int len=in.read(buffer);

System.out.println("ip"+inetAddress.getHostAddress()+new String(buffer,
0, len));

//4.服务端，要向客户端反馈数据
OutputStream out = socket.getOutputStream();
out.write("强哥，今天天气很好，我们去爬山".getBytes());

//5.关闭资源
socket.close();
ss.close();

```

```

// Socket(InetAddress address, int port)
// 1.创建Socket对象
Socket socket = new Socket("127.0.0.1", 8080);

//2.发送数据
//获取字节输出流
OutputStream out = socket.getOutputStream();

out.write("林老师，我来了".getBytes());

//3.获取服务器端响应的数据
InputStream in = socket.getInputStream();

byte[] buffer=new byte[1024];
int len=in.read(buffer);
System.out.println(new String(buffer, 0, len));

socket.close();

```

3.4 上传视频的客户端

```
package com.gxa.tcp;

import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.net.Socket;

//上传文件的客户端
public class UploadClient {

    public static void main(String[] args) throws Exception, IOException {

        //1.创建Socket对象
        Socket socket=new Socket("127.0.0.1", 8080);

        //2.从磁盘读取数据到内存 in
        //BufferedInputStream(InputStream in) 字节输入缓冲流
        BufferedInputStream in = new BufferedInputStream(new
        FileInputStream("C:\\Users\\gxa\\Desktop\\001.jpg"));

        //3.往服务器写出数据
        BufferedOutputStream out = new
        BufferedOutputStream(socket.getOutputStream());

        //4.流的模板代码
        byte[] buffer=new byte[1024];

        int len=0;

        while((len=in.read(buffer))>0) {
            out.write(buffer,0,len);
        }

        //5.传输文件，传输完毕，关闭输出流
        socket.shutdownOutput();

        //6.获取服务器反馈
        InputStream ii = socket.getInputStream();

        byte[] bys=new byte[1024];

        int l=ii.read(bys);

        System.out.println(new String(bys, 0, l));

        in.close();

        socket.close();
    }
}
```

```
}  
  
}
```

3.5 上传视频的服务端

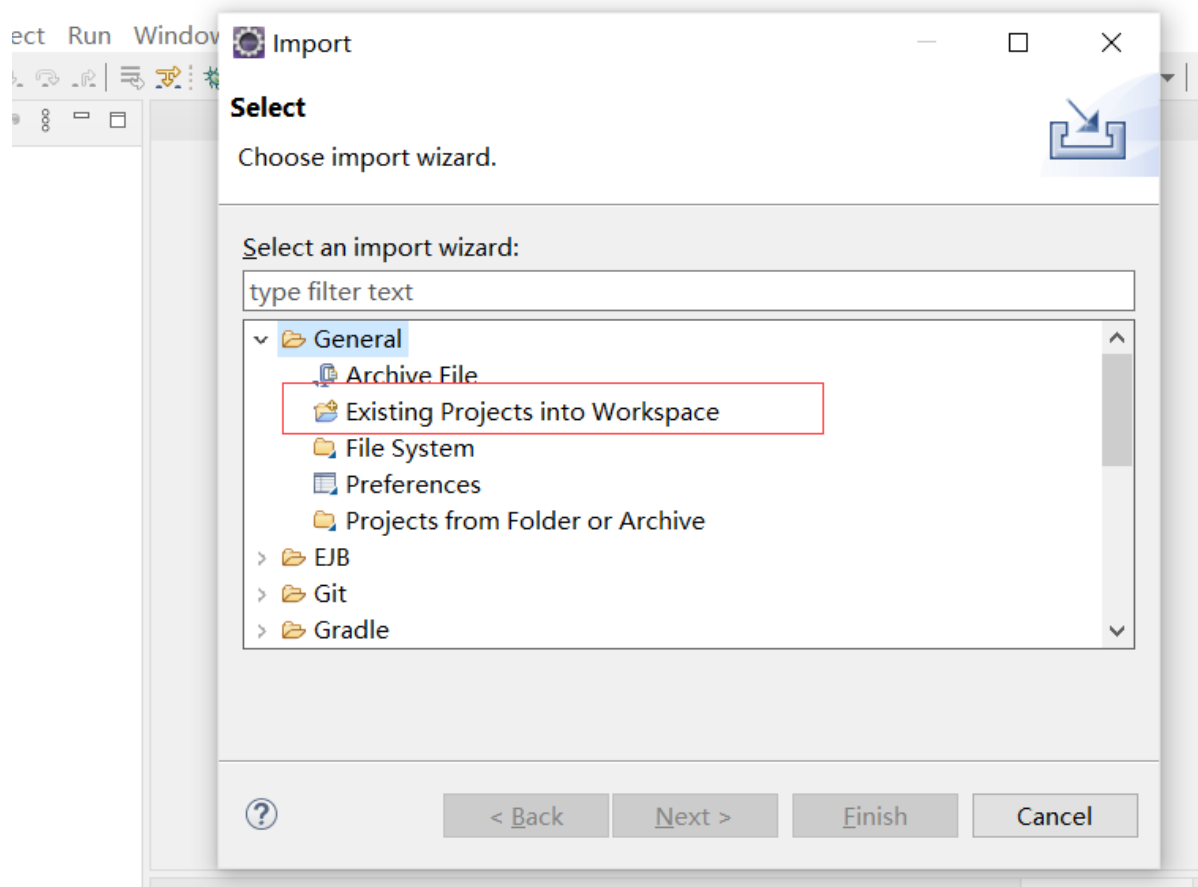
```
package com.gxa.tcp;  
  
import java.io.BufferedInputStream;  
import java.io.BufferedOutputStream;  
import java.io.FileOutputStream;  
import java.net.ServerSocket;  
import java.net.Socket;  
  
public class UploadServer {  
  
    public static void main(String[] args) throws Exception {  
  
        //1.创建ServerSocket对象  
        ServerSocket serverSocket=new ServerSocket(8080);  
  
        //2.获取到客户端Socket对象  
        Socket socket = serverSocket.accept();  
  
        //3.读取客户端的数据  
        BufferedInputStream in = new  
        BufferedInputStream(socket.getInputStream());  
  
        //4.往服务器磁盘写出数据  
        BufferedOutputStream out=new BufferedOutputStream(new  
        FileOutputStream("001.jpg"));  
  
        //5.流的模板代码  
        byte[] buffer=new byte[1024];  
  
        int len=0;  
  
        while((len=in.read(buffer))>0) {  
            out.write(buffer,0,len);  
        }  
  
        //5.反馈服务器端已经上传成功!  
        socket.getOutputStream().write("文件上传成功!".getBytes());  
    }  
}
```

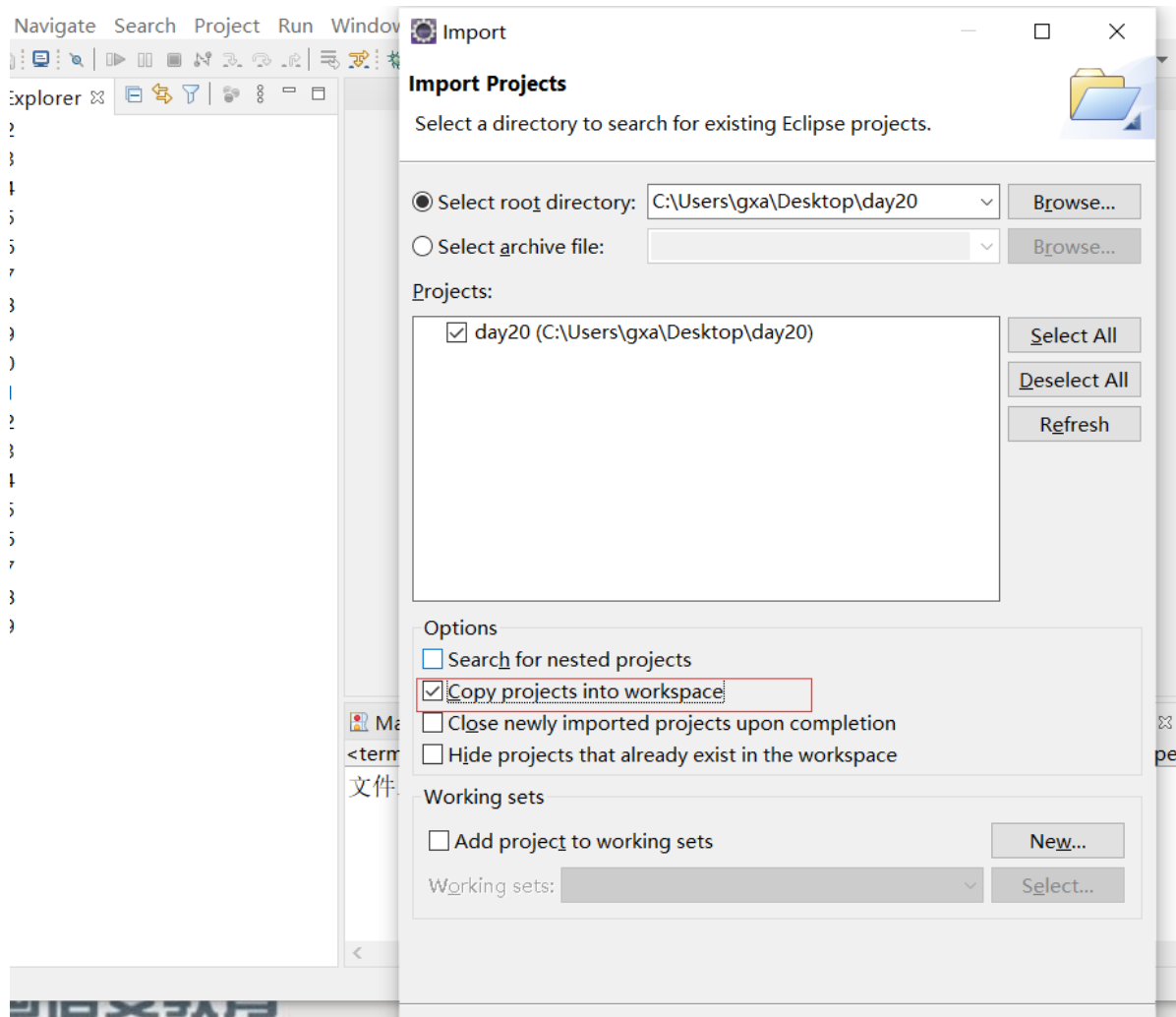
```
out.close();  
socket.close();  
serverSocket.close();
```

```
}
```

```
}
```

3.6 导入项目





4、三次握手和四次挥手

